# Project 1: Tensor Implementation in C++

## 1 Introduction

In class, we implemented a two-dimensional `Matrix<T>` class. In this project, you will extend this idea and implement a generic `Tensor<T>` class that supports data with an arbitrary number of dimensions.

For example:

```
Tensor<int> a({10});          // 1D tensor
Tensor<float> b({3, 4});        // 2D tensor
Tensor<double> c({3, 32, 32});   // 3D tensor
```

A tensor should store its shape and data using STL containers, such as `std::vector`. The project focuses on class templates, STL containers, multi-dimensional data representation, indexing, and operator overloading.

## 2 Tasks

Implement a class template:

```
template <typename T>
class Tensor {
private:
    std::vector<std::size_t> shape_;
    std::vector<T> data_;

public:
    Tensor();
    Tensor(const std::vector<std::size_t>& shape);

    std::size_t ndim() const;
    std::size_t size() const;
    const std::vector<std::size_t>& shape() const;

    T& at(const std::vector<std::size_t>& indices);

    const T& at(const std::vector<std::size_t>& indices) const;

    void reshape(const std::vector<std::size_t>& newShape);

    void flatten();

    Tensor<T> operator+(const Tensor<T>& other) const;

    Tensor<T> operator-(const Tensor<T>& other) const;

    Tensor<T> operator*(const Tensor<T>& other) const;  // element-wise
```

```
    T sum() const;
    double mean() const;

    void printInfo() const;
};
```

Your implementation should include the following functionality:

1. **Tensor Construction and Storage**

   Store tensor elements in a one-dimensional `std::vector<T>`. The tensor shape should be stored separately. Multi-dimensional indices should be mapped to positions in this underlying one-dimensional storage.

   For example:

   ```
   Tensor<double> A({2, 3, 4});
   ```

   represents a tensor with shape $2 \times 3 \times 4$ and 24 elements.

2. **Shape Information**

   Provide functions to retrieve basic information about a tensor. For example, for the tensor `A`:

   - `A.ndim()` returns the number of dimensions: `3`
   - `A.size()` returns the total number of elements: `24`
   - `A.shape()` returns the complete tensor shape: `{2, 3, 4}`

3. **Multi-Dimensional Indexing**

   Support reading and modifying tensor elements using multi-dimensional indices:

   ```
   A.at({1, 2, 3}) = 5.0;
   double x = A.at({1, 2, 3});
   ```

   Your implementation should convert the multi-dimensional index into the corresponding index in the underlying `std::vector<T>` storage and detect invalid indices. Invalid indices should be detected.

4. **Reshape and Flatten**

   Implement operations to change the tensor shape without changing its data:

   ```
   A.reshape({6, 4});
   A.flatten();
   ```

   A reshape operation is valid only when the total number of elements remains unchanged.

5. **Tensor Operations**

   Implement at least the following element-wise operation:

   ```
   Tensor<T> operator+(const Tensor<T>& other) const;
   Tensor<T> operator-(const Tensor<T>& other) const;
   Tensor<T> operator*(const Tensor<T>& other) const;

   T sum() const;
   double mean() const;
   ```

Element-wise operations between tensors require compatible shapes.

6. **Tensor Information**

   Provide a function such as `printInfo()` to display basic tensor information. For example:

```
Shape: [3, 32, 32]
Dimensions: 3
Elements: 3072
```

You may introduce additional private data members, helper functions, or alternative interfaces when appropriate. Clearly explain important design decisions in your report.

# 3 Submission Requirements

- **Source Code (35 pts)**: Submit all source files, including `.cpp`, `.h`, and `.tpp` files if applicable. The code must compile and run without modification.
- **Test Code (15 pts)**: Testing is an essential part of software development. Provide a `test.cpp` that **you design yourself** to verify the correctness of the implemented features.

  Your tests should include tensors with different dimensions and data types, and should test indexing, reshape/flatten, tensor operations, and invalid inputs. The tests should be clear and meaningful, rather than only printing values.
- **Report (at least 2 pages) (30 pts)**: Upload a **PDF** report that includes:

  1. **Implementation Logic**: Describe how your Tensor is represented, including shape, data storage, and multi-dimensional indexing.
  2. **Test Design**: Explain how you designed tests to verify the correctness of the required functionality and why the tests are sufficient.
  3. **Group Work Distribution**: Clearly describe each member's contribution. All members receive the same grade, so the work should be shared equitably.
  4. **Challenges and Solutions**: Summarize the main difficulties encountered and how you resolved them.
- **Video Demo (5–8 minutes) (20 pts)**: Briefly explain the key components of your implementation and run representative test cases to demonstrate that your Tensor class works correctly. Each group member should present and explain the part of the project for which they were primarily responsible.